



Seminario de Programación



C++ - STL

Patricio Olivares - Wladimir Ormazábal

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Namespace

Namespace

- En C++, la declaración de un **namespace** es un bloque de código que permite agrupar elementos (clases, objetos y funciones) de un programa.
- Los namespace permiten agrupar el código en bloques lógicos y evitar colisiones de nombres.
- Sintaxis (puede ser en varios archivos):

```
namespace identificador{  
    // Acá van las declaraciones, clases, objetos y funciones  
}
```

- Para acceder a los elementos dentro del namespace se utiliza el operador ::

Namespace

Namespace

- Ejemplo:

```
namespace espacio{  
    int a,b;  
}  
// ...  
// Accediendo a las variables del espacio  
espacio::a;  
espacio::b;
```

C++: Clases STL

- La STL (Standard Template Library) es un conjunto de datos genéricos y algoritmos escritos en C++.
- Define clases para manejar strings, arreglos, conjuntos, listas, colas, etc. sin importar el tipo de dato (templates).
- Los algoritmos de búsqueda y ordenamiento se encuentran optimizados para estos contenedores.
- Estos se encuentran definidos en el namespace **std**.

Namespace y llamadas a clases STL

Para poder acceder a clases y métodos de otras bibliotecas como STL, es necesario indicarle al compilador desde dónde provienen.

Namespace y llamadas a clases STL

Para poder acceder a clases y métodos de otras bibliotecas como STL, es necesario indicarle al compilador desde dónde provienen.

- Se debe indicar en el encabezado a través de **include** las bibliotecas a utilizar

Namespace y llamadas a clases STL

Para poder acceder a clases y métodos de otras bibliotecas como STL, es necesario indicarle al compilador desde dónde provienen.

- Se debe indicar en el encabezado a través de **include** las bibliotecas a utilizar

```
In [ ]: // Permite usar las clases y métodos declarados .  
        #include <string>  
        #include <iostream>
```

Namespace y llamadas a clases STL

Namespace y llamadas a clases STL

Para acceder a los elementos se debe indicar **a qué namespace pertenecen** (si aplica). Existen dos formas:

Namespace y llamadas a clases STL

Para acceder a los elementos se debe indicar **a qué namespace pertenecen** (si aplica). Existen dos formas:

- Incluyendo la declaración **using namespace**

Namespace y llamadas a clases STL

Para acceder a los elementos se debe indicar **a qué namespace pertenecen** (si aplica). Existen dos formas:

- Incluyendo la declaración **using namespace**

```
In [ ]: // Declarar uso de namespace existente :  
using namespace std;  
  
// ...  
string nombre = "Juan";  
cout << nombre << endl;
```

Namespace y llamadas a clases STL

Para acceder a los elementos se debe indicar **a qué namespace pertenecen** (si aplica). Existen dos formas:

- Incluyendo la declaración **using namespace**

```
In [ ]: // Declarar uso de namespace existente :  
        using namespace std;  
  
        // ...  
        string nombre = "Juan";  
        cout << nombre << endl;
```

- Utilizando como prefijo el nombre del namespace y dos **dos puntos (::)**

Namespace y llamadas a clases STL

Para acceder a los elementos se debe indicar **a qué namespace pertenecen** (si aplica). Existen dos formas:

- Incluyendo la declaración **using namespace**

```
In [ ]: // Declarar uso de namespace existente :  
        using namespace std;  
  
        // ...  
        string nombre = "Juan";  
        cout << nombre << endl;
```

- Utilizando como prefijo el nombre del namespace y dos **dos puntos (::)**

```
In [ ]: std::string nombre = "Juan";  
        std::cout << nombre << std::endl;
```

Classes STL - Strings

Clases STL - Strings

- La clase **string** provee soporte para arreglos de caracteres.
- Agrega adicionalmente métodos de manejo de caracteres tales como **empty**, **length**, **append**, **find** y acceso directo con []

Clases STL - Strings

Ejemplo

Clases STL - Strings

Ejemplo

```
In [ ]: #include <string>
#include <iostream>
using namespace std;
int main(){
    string a("abcd efg");
    string b("xyz ijk");
    string c;
    cout << a << " " << b << endl ; // Salida : abcd efg xyz ijk
    cout << " String vacío: " << c.empty() << endl ; // String vacío: 1
    c = a + b; // Concatenación
    cout << c << endl; // Salida: abcd efgxyz ijk
    cout << "Largo: " << c.length() << endl; // Largo: 15
    string d = c; // Copia
    cout << d << endl; // Salida: abcd efgxyz ijk
    cout << "Primer caracter: " << c[0] << endl; // Salida: a
    string f("Otra forma de inicializar...");
    cout << "String f: " << f.append("ZZZ") << endl ; // Salida: String f
    cout << "Find: " << f.find("for") << endl; // Find: 5. Si no encuentra
```

Clases STL - Vector (arreglo)

Clases STL - Vector (arreglo)

- Clase para arreglos dinámicos en la STL.
- Funciona con memoria contigua, agregando bloques de memoria en forma transparente al programador.
- La inserción de un elemento al medio es lenta, pero la inserción de varios elementos al mismo tiempo es muy rápida,
- Métodos principales: **push_back**, **resize**, **reserve**, **capacity**, acceso directo con `[]`, **clear** y **size**
- Agrega elementos de manejo de memoria

Clases STL - Vector (arreglo)

Clases STL - Vector (arreglo)

```
In [ ]: // Instanciación vacía:
std::vector<double> a1;
// Instanciación con 4 elementos:
std::vector<double> a2(4);
a2.push_back(2.5);
a2.push_back(3);
a2.push_back(5);
for(int i=0; i<a2.size(); i++)
    std::cout << a2[i] << std::endl;

// Chequea la capacidad de memoria ( siempre mayor que size ):
std::cout << a3.capacity();
// Reescalar tamaño del arreglo. Elimina último elemento de a2:
a2.resize(3);
// Reescalar tamaño del arreglo. Agrega 10 elementos a a1 (constructor):
a1.resize(10);
// Reescalar capacidad del arreglo (memoria). Aumenta el tamaño de la memoria
// 20 elementos en a1 (constructor). No crea los objetos:
```



```
a1.reserve(20);  
// Reescalar capacidad del arreglo (memoria).  
// Ningún efecto en a3, pues nuevo valor < que capacidad actual:  
a3.reserve(3);
```

Clases STL - Vector (arreglo)

```
In [ ]: // Instanciación vacía:
std::vector<double> a1;
// Instanciación con 4 elementos:
std::vector<double> a2(4);
a2.push_back(2.5);
a2.push_back(3);
a2.push_back(5);
for(int i=0; i<a2.size(); i++)
    std::cout << a2[i] << std::endl;

// Chequea la capacidad de memoria ( siempre mayor que size ):
std::cout << a3.capacity();
// Reescalar tamaño del arreglo. Elimina último elemento de a2:
a2.resize(3);
// Reescalar tamaño del arreglo. Agrega 10 elementos a a1 (constructor):
a1.resize(10);
// Reescalar capacidad del arreglo (memoria). Aumenta el tamaño de la mem
// 20 elementos en a1 (constructor). No crea los objetos:
```

```
a1.reserve(20);  
// Reescalar capacidad del arreglo (memoria).  
// Ningún efecto en a3, pues nuevo valor < que capacidad actual:  
a3.reserve(3);
```

- Si se acaba el espacio reservado al agregar más elementos, vector aumenta la capacidad a **2*capacity()**.

Classes STL - Map

Clases STL - Map

- Contenedor asociativo que almacena elementos como una combinación de una **llave** y un valor.
- Las llaves se utilizan para ordenar e identificar de forma **única** los elementos.
- Usan internamente una implementación de Red-Black Tree (curso Estructura de Datos y Algoritmos).
- Los valores pueden ser accedidos directamente por su respectiva llave usando el operador `[]`.

Classes STL - Map

Clases STL - Map

```
In [ ]: #include <map>
#include <iostream>
using namespace std;
int main(){
    // Declaración <clave, valor>
    map <string, double> lista_notas;
    lista_notas["Pedro"] = 3.5;
    lista_notas["Ana"] = 4.6;
    lista_notas["Jose"] = 4.7;
    lista_notas["Maria"] = 3.6;
    // Imprimir nota de Maria:
    cout << lista_notas["Maria"] << endl; // Salida: 3.6
    // Borrar según clave (borra a maria):
    lista_notas.erase("Maria");
    cout << " Largo : " << lista_notas.size() << endl; // Largo :3
    // Find retorna iterador si encuentra y grade_list.end() sino
    if(lista_notas.find("Juan") == lista_notas.end())
        std :: cout << "Juan no esta !" << endl;
    // Iterator: Recorrer secuencialmente la lista
    map <string, double>::iterator iter;
    for(iter = lista_notas.begin(); iter != lista_notas.end(); iter++)
        // Derreferencia iterador y muestra cada para "llave: valor" (ej.
```



```
        cout << (*iter).first << ":" << (*iter).second << endl;
// Borra lista
lista_notas.clear();
return 0;
}
```

Clases STL - otros

Clases STL - otros

Dentro de la clase STL, existen muchos más contenedores y funcionalidades de las presentadas. Algunas de ellas son:

- **set**: conjunto ordenado, provee operaciones de conjuntos
- **stack**: Implementa una pila (LIFO: Last In First Out)
- **queue**: Implementa una cola (FIFO: First In First Out)
- **multiset**: igual que set, pero acepta llaves duplicadas
- **multimap**: igual que map, pero acepta llaves duplicadas.

Referencias

Referencias

- <https://www.cplusplus.com/reference/stl/> (documentación de stl, revisar para más profundidad!)
- <https://docs.microsoft.com/en-us/cpp/cpp/namespaces-cpp>
- <https://ccia.ugr.es/~jfv/ed1/c++/cdrom4/paginaWeb/stl.htm>
- <https://web.eecs.utk.edu/~bvanderz/cs302/notes/stl.html>

